Université Mohammed Ier
Faculté des Sciences
Département d'Informatique.
Oujda - MAROC -

Année universitaire 2025/2026
Filières : DI & GI (Semestre 4)
Module : POO (Langage C++)

Série TP N° 3

Exercice 1 : Classe avec membre statique

Ecrire un programme en langage C++ permettant de définir et d'utiliser une classe intitulée ***CompteBancaire***. Cette classe doit comporter les éléments suivants :

1) Un **membre de données statique** nommé ***dernierNumero***, utilisé pour générer des numéros de compte uniques.
2) Un **membre de données** nommé ***numero***, propre à chaque objet de la classe.
3) Un **constructeur** qui attribue automatiquement à chaque nouveau compte un numéro unique, obtenu en incrémentant la valeur de ***dernierNumero***.
4) Une méthode ***afficherNumero()*** chargée d'afficher le numéro du compte.

La variable statique ***dernierNumero*** doit être initialisée à la valeur ***2000***.
Dans la fonction principale du programme, vous devez :

- créer trois objets de type ***CompteBancaire*** ;
- afficher le ***numéro*** attribué à chacun des trois comptes.

Exercice 2 : Utilisation d'un membre statique

On souhaite illustrer l'utilisation d'un **membre statique** dans une classe afin de gérer un **taux d'intérêt commun à l'ensemble des comptes d'épargne**. Pour cela, écrire un programme en **langage C++** permettant de réaliser les tâches suivantes :

1) Définir une classe nommée `CompteEpargne` comportant :
   - Un **membre de données statique** de type réel nommé `tauxInteret`, représentant le taux d'intérêt commun appliqué à tous les comptes.Ce taux doit être initialisé à **5%**.
2) Implémenter dans cette classe :
   - Une **méthode statique** nommée `modifierTaux(float nouveauTaux)` permettant de modifier la valeur du taux d'intérêt.
   - Une **méthode statique** nommée `afficherTaux()` permettant d'afficher la valeur actuelle du taux d'intérêt.
3) Dans la fonction principale `main()` :
   - Afficher la valeur initiale du taux d'intérêt.
   - Modifier ce taux à **6,5 %** à l'aide de la méthode appropriée.
   - Afficher ensuite la **nouvelle valeur du taux d'intérêt**.

Exercice 3 : Surcharge de fonctions

Dans le cadre de l'étude de la **surdéfinition de fonctions**, on souhaite concevoir un programme permettant de calculer le volume de différentes formes géométriques. Pour cela :

1. Définir une classe nommée `Volume` contenant une fonction `calculerVolume()` surchargée afin de permettre le calcul du volume des solides suivants :
   a. un **cube**, dont le volume est déterminé à partir de la valeur de son **côté** ;
   b. un **cylindre**, dont le volume est calculé à partir de son **rayon** et de sa **hauteur** ;
   c. un **parallélépipède rectangle**, dont le volume est déterminé à partir de sa **longueur**, sa **largeur** et sa **hauteur**.
2. Dans la fonction principale `main()` :
   a. Déclarer un objet `v` de type `Volume` ;
   b. Utiliser les méthodes appropriées pour calculer et afficher les volumes suivants :

i. un **cube** de côté égal à **3** ;
ii. un **cylindre** de **rayon 2** et de **hauteur 5** ;
iii. un **parallélépipède rectangle** de **longueur 2, largeur 3** et **hauteur 4**.

Exercice 4 : Utilisation du pointeur `this`

Dans le but d'illustrer l'utilisation du **pointeur d'autoréférence `this` en langage C++**, écrire un programme répondant aux spécifications suivantes :

1. Définir une classe nommée `Objet` contenant une **fonction membre** appelée `getThis()`, dont le rôle est de **retourner un pointeur vers l'objet courant** en utilisant le pointeur implicite `this`.
2. Dans la fonction principale `main()` :
   a. Créer un objet de la classe `Objet`.
   b. Afficher **l'adresse mémoire de cet objet** en utilisant **l'opérateur d'adresse &**.
   c. Appeler la méthode `getThis()` et afficher **l'adresse retournée**.
3. Comparer les deux adresses affichées afin de **mettre en évidence le rôle du pointeur `this`**, qui correspond à l'adresse de l'objet courant.

Exercice 5 : Constructeurs et destructeurs

Écrire un programme en **langage C++** illustrant l'utilisation des **constructeurs, destructeurs et méthodes membres** dans une classe. Pour cela, réaliser les étapes suivantes :

1. Définir une classe nommée `Rectangle` comportant :
   a. Deux **attributs privés** de type `int` : `largeur` et `hauteur`, représentant respectivement la largeur et la hauteur du rectangle.
   b. Un **constructeur** permettant d'initialiser ces deux attributs.
   c. Un **destructeur**.
   d. Une **méthode membre** nommée `calculerAire()` qui retourne l'aire du rectangle.
   e. Une **méthode membre** nommée `afficher()` permettant d'afficher les dimensions ainsi que l'aire du rectangle.
2. Dans la fonction principale `main()` :
   a. Créer un objet `R1` de la classe `Rectangle`, initialisé avec les dimensions **(4, 5)**.
   b. À l'intérieur d'un **bloc**, créer un second objet automatique `R2` initialisé avec les dimensions **(6, 7)**.
   c. Calculer et afficher l'aire de chacun des deux rectangles.
3. Observer le comportement du **destructeur** lors de la sortie du bloc pour l'objet `R2`, puis à la fin du programme pour l'objet `R1`.

Exercice 6 : Tableaux d'objets

On souhaite illustrer l'utilisation des **classes, des constructeurs, des destructeurs et des tableaux d'objets** en **langage C++**. Pour cela, écrire un programme répondant aux spécifications suivantes :

1. Définir une classe nommée `Personne` comportant :
   a. un **attribut nom** de type `string` ;
   b. un **attribut age** de type `int`.
2. Implémenter dans cette classe :
   a. un **constructeur** permettant d'initialiser les attributs `nom` et `age` ;
   b. un **destructeur** qui affiche le **nom de l'objet détruit**, afin d'identifier l'objet concerné lors de sa destruction ;
   c. une **méthode d'affichage** permettant d'afficher les informations relatives à l'objet, notamment **le nom et l'âge**.
3. Dans la fonction principale `main()` :
   a. déclarer un **tableau `T` d'objets de type `Personne`** (objets automatiques) ;
   b. afficher les informations de chaque objet du tableau en utilisant la méthode d'affichage.